

异常控制流

一个程序的正常执行流程有两种顺序：一种是按指令存放的顺序执行，即新的 PC 值为当前指令地址加当前指令长度；一种是跳转到由转移类指令指出的转移目标地址处执行，即新的 PC 值为转移目标地址。CPU 所执行的指令的地址序列称为CPU 的控制流，通过上述两种方式得到的控制流为正常控制流。

在程序正常执行过程中，CPU 会因为遇到内部异常事件或外部中断事件而打断原来程序的执行，转去执行操作系统提供的针对这些特殊事件的处理程序。这种由于某些特殊情况引起用户程序的正常执行被打断所形成的意外控制流称为异常控制流（Exceptional Control of Flow, ECF）。显然，计算机系统必须提供一种机制，使得自身能够实现异常控制流。

在计算机系统的各个层面都有实现异常控制流的机制。例如，在底层的硬件层，CPU 中有检测异常和中断事件并将控制转移到操作系统内核执行的机制；在中间的操作系统层，内核能通过进程的上下文切换将一个进程的执行转移到另一个进程执行；在上层的应用软件层，一个进程可以直接发送信号到另一个进程，使得接收到信号的进程将控制转移到它的一个信号处理程序执行。

本章主要介绍硬件层和操作系统层中涉及的对于内部异常和外部中断的异常控制流实现机制。主要内容包括：进程与进程上下文切换，异常的类型、异常的捕获和处理、中断的捕获和处理，系统调用的实现机制等。

7.1 进程与进程的上下文切换

7.1.1 程序和进程的概念

任何一个应用问题描述为处理算法后，都要用某种编程语言表示出来。绝大多数情况下，都采用高级语言编写源程序，而高级语言源程序需要进行编译转换为目标程序，在链接之前的目标程序是可重定位目标程序形式，链接之后是可执行目标形式，其代码部分是一个机器指令序列，可以被计算机直接执行。对计算机来说，程序（program）就是代码和数据的集合，程序的代码是一个机器指令序列，因而程序是一种静态的概念。它可以作为目标模块存放在磁盘中，或者作为一个存储段存在于一个地址空间中。

简单来说，进程（process）就是程序的一次运行过程。更确切地说，进程是一个具有一定独立功能的程序关于某个数据集合的一次运行活动，因而进程具有动态的含义。计算机处理的

所有**任务**实际上是由进程完成的。

每个应用程序在系统中运行时（用户进程）均有属于自己的存储空间，用来存储自己的程序代码和数据，包括只读区（代码和只读数据）、可读可写数据区（初始化数据和未初始化数据）、动态的堆区和栈区等。

进程是操作系统对处理器中程序运行过程的一种抽象。进程有自己的生命周期，它由于任务的启动而创建，随着任务的完成（或终止）而消亡，它所占用的资源也随着进程的终止而释放。

一个可执行目标文件可以被多次加载执行，也就是说，一个程序可能对应多个不同的进程。例如，用 word 程序编辑一个文档时，相应的进程就是 winword.exe，如果多次启动同一个 word 程序，就得到多个 winword.exe 进程。

小贴士

用户指定的任务与计算机系统中的任务不是同一个概念，计算机系统中的任务通常就是指进程。例如，Linux 内核中通常把进程称为任务，每个进程主要通过一个称为**进程描述符**（process descriptor）的结构来描述，其结构类型定义为 task_struct，包含了一个进程的所有信息。所有进程通过一个双向循环链表实现的**任务列表**（task list）来描述，任务列表中的每个元素是一个进程描述符。IA-32 中的任务状态段（TSS）、任务门（task gate）等概念中所称的任务，实际上也是指进程。

对于现代多任务操作系统，通常一段时间内会有多个不同的进程在系统中运行，这些进程轮流使用处理器并共享同一个主存储器。但是，程序员在编写程序或者语言处理系统在编译并链接生成可执行目标文件时，并不用考虑如何和其他程序一起共享处理器和存储器资源，而只要考虑自己的程序代码和所用数据如何组织在一个独立的虚拟存储空间中。也就是说，程序员和语言处理系统可以把一台计算机的所有资源看成是由自己的程序独占，都以为自己的程序是在处理器上执行的和在存储空间中存放的唯一的用户程序。显然，这是一种“错觉”。这种“错觉”带来了极大的好处，它简化了程序员的编程以及语言处理系统的处理，即简化了编程、编译、链接、共享和加载等整个过程。

“进程”的引入为应用程序提供了以下两方面的抽象：一个独立的逻辑控制流和一个私有的虚拟地址空间。每个进程拥有一个独立的逻辑控制流，使得程序员以为自己的程序在执行过程中独占使用处理器；每个进程拥有一个私有的虚拟地址空间，使得程序员以为自己的程序在执行过程中独占存储器。

为了实现上述两个方面的抽象，操作系统必须提供一整套的管理机制，包括处理器调度、进程的上下文切换、虚拟存储管理等。

7.1.2 进程的逻辑控制流

一个可执行目标文件被加载并启动执行后，就成为一个进程。不管是静态链接生成的完全链接可执行文件，还是动态链接后在存储器中形成的完全链接可执行目标，它们的代码段中的每条指令都有一个确定的地址，在这些指令的执行过程中，会形成一个指令执行的地址序列，

对于确定的输入数据，其指令执行的地址序列也是确定的。这个确定的指令执行地址序列称为进程的逻辑控制流。

对于一个具有单处理器核的系统，如果在一段时间内有多个进程在其上运行，那么，这些进程会轮流使用处理器，也即处理器的物理控制流由多个逻辑控制流组成。例如，假定在某段时间内，单处理器系统中有三个进程 p_1 、 p_2 和 p_3 在运行，其运行轨迹如图 7.1 所示。图中水平方向为时间，垂直方向为执行指令的地址。

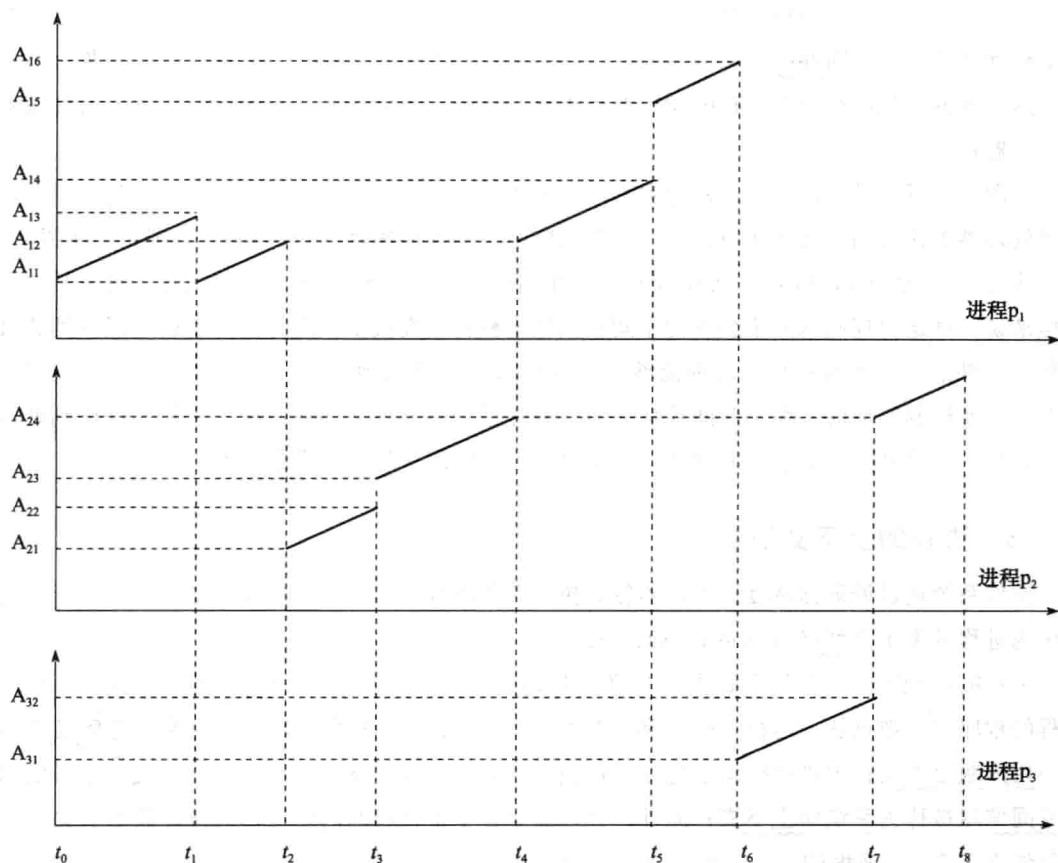


图 7.1 进程 p_1 、 p_2 和 p_3 的逻辑控制流

在图 7.1 中，进程 p_1 的执行过程为：从 t_0 到 t_1 时刻按序执行地址 A_{11} 到 A_{13} 处的指令，然后再跳转到 A_{11} 开始按序执行，直到 t_2 时刻执行到 A_{12} 处指令时被换下处理器，一直等到 t_4 时刻，又从上次被中断的 A_{12} 处被换上处理器开始执行，直到 t_6 时刻执行完成。一个进程逻辑控制流总是确定的，不管中间是否被其他进程打断，也不管被打断几次或在哪里被打断，这样，就可以保证一个进程的执行不管怎么被打断其行为总是一致的。可以看出，进程 p_1 的逻辑控制流为 $A_{11} \sim A_{13}$ 、 $A_{11} \sim A_{14}$ 、 $A_{15} \sim A_{16}$ 。也即其执行轨迹总是先按序从 A_{11} 执行到 A_{13} ；然后从 A_{13} 跳到 A_{11} ，按序从 A_{11} 执行到 A_{14} ；再从 A_{14} 跳到 A_{15} ，按序从 A_{15} 执行到 A_{16} 。在 p_1 整个逻辑控制流中，控制流在 A_{12} 处被 p_2 打断了一次。

进程 p_2 在 t_2 时刻被换上执行，在 t_4 时刻被换下处理器，然后在 t_7 时刻再次被换上处理器

执行,直到 t_8 时刻执行完成。在 p_2 整个逻辑控制流中,控制流在 A_{24} 处被 p_1 打断了一次。

进程 p_3 则在 t_6 时刻被换上处理器执行,到 t_7 时刻执行完成。在 p_3 整个逻辑控制流中没有被打断。

从图 7.1 可以看出,有些进程的逻辑控制流在时间上有交错,通常把这种不同进程的逻辑控制流在时间上交错或重叠的情况称为**并发** (concurrency)。例如,进程 p_1 和 p_2 的逻辑控制流在时间上是交错的,因此,进程 p_1 和 p_2 是并发运行的,同样, p_2 和 p_3 也是并发的,而 p_1 和 p_3 不是并发的。并发执行的概念与处理器核数没有关系,只要两个逻辑控制流在时间上有交错或重叠都称为并发,而**并行** (parallelism) 则是并发执行的一个特例,即并行执行的两个进程一定是并发的。我们称两个进程的逻辑控制流是并行的,是指它们并发地运行在不同的处理器或处理器核上。

从图 7.1 可以看出,三个进程的逻辑控制流在同一个时间轴上串行,也即进程是轮流在一个单处理器上执行的。连续执行同一个进程的时间段称为**时间片** (time slice)。例如,在图 7.1 中,从 t_0 到 t_2 为一个时间片,从 t_2 到 t_4 为一个时间片,从 t_4 到 t_6 为一个时间片。对于某一个进程来说,其逻辑控制流并不会因为中间被其他进程打断而改变,因为被打断后还能回到原被打断的“断点”处继续执行。这种能够从被其他进程打断的地方继续执行的功能是由进程的上下文切换机制实现的。每个时间片结束时,通过进程的上下文切换,换一个新的进程到处理器上执行,从而开始一个新的时间片,这个过程称为**时间片轮转处理器调度**。

7.1.3 进程的上下文切换

操作系统通过处理器调度让处理器轮流执行多个进程。实现不同进程中指令交替执行的机制称为进程的**上下文切换** (context switching)。

进程的物理实体(代码和数据等)和支持进程运行的环境合称为**进程的上下文**。由用户进程的程序块、数据块、运行时的堆和用户栈(统称为**用户堆栈**)等组成的**用户空间信息**被称为**用户级上下文**;由进程标识信息、进程现场信息、进程控制信息和系统内核栈等组成的**内核空间信息**被称为**系统级上下文**;此外,处理器中各个寄存器的内容被称为**寄存器上下文**(也称**硬件上下文**)。进程的上下文包括了用户级上下文、系统级上下文和寄存器上下文。其中,用户级上下文地址空间和系统级上下文地址空间一起构成了一个进程的整个存储器映像,如图 7.2 所示,实际上它就是进程的虚拟地址空间。**进程控制信息**包含各种内核数据结构,例如,记录有关进程信息的进程表(process table)、页表、文件表等。

上下文切换发生在操作系统调度一个新进程到处理器上运行时,它需要完成以下三件事:①将当前进程的寄存器上下文保存到当前进程的系统级上下文的现场信息中;②将新进程系统级上下文中的现场信息作为新的寄存器上下文恢复到处理器的各个寄存器中;③将控制转移到新进程执行。这里,一个重要的上下文信息是 PC 的值,当前进程被打断的断点处的 PC 作



图 7.2 进程的存储映像

为寄存器上下文的一部分被保存在进程现场信息中，这样，下次该进程再被调度到处理器上执行时，就可以从其现场信息中获得断点处的 PC，从而能从断点处开始执行。

下面给出的例子是一种典型的进程上下文切换场景。以下是经典的 hello.c 程序：

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("hello, world\n");
6 }
```

对于上述高级语言源程序，首先，需先对其进行预处理并编译成汇编语言程序，然后再用汇编程序将其转换为可重定位的二进制目标程序，再和库函数目标模块 printf.o 进行链接，生成最终的可执行目标文件 hello。

假定在 UNIX 系统上启动 hello 程序，其 shell 命令行和 hello 程序运行的结果如下。

```
unix> ./hello [Enter]
hello, world
unix>
```

上下文切换指把正在运行的进程换下，换一个新进程到处理器执行。图 7.3 给出了上述 shell 命令行执行过程中 shell 进程和 hello 进程的上下文切换过程。首先运行 shell 进程，从 shell 命令行中读入字符串“./hello”到主存；当 shell 进程读到字符“[Enter]”后，转到操作系统执行，由操作系统进行上下文切换，以保存 shell 进程的上下文并创建 hello 进程的上下文；hello 进程执行结束后，再转到操作系统完成将控制权从 hello 进程交回给 shell 进程的切换。

从上述过程可以看出，在一个进程的整个生命周期中，可能会有其他不同的进程在处理器中交替运行。例如，对于图 7.3 中的 hello 进程的执行，用户感觉到的时间还包括了操作系统执行上下文切换的时间。对于图 7.1 所示的 p_1 进程，用户感觉到的时间除了包括了操作系统执行上下文切换的时间外，还包括了用户进程 p_2 的一段执行时间。因此，对于每个进程的运行很难凭感觉测量出准确时间。

显然，处理器调度等事件会引起用户进程的正常执行被打断，因而形成了突变的异常控制流，而进程的上下文切换机制很好地解决了这类异常控制流，实现了从一个进程安全切换到另一个进程执行的过程。

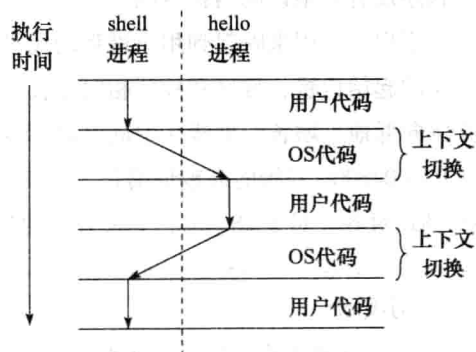


图 7.3 进程上下文切换示例

7.1.4 进程的私有地址空间

“进程”的引入除了为应用程序提供了一个独立的逻辑控制流，还为应用程序提供了一个私有的地址空间，使得程序员以为自己的程序在执行过程中独占拥有存储器，这个私有地址空

间就是第6章介绍的虚拟地址空间。例如，图7.4给出了在Intel架构下Linux操作系统中的一个进程对应的虚拟地址空间映像。

如图7.4所示，整个虚拟地址空间分为两大部分：内核虚拟存储空间（简称内核空间）和进程虚拟存储空间（简称用户空间）。在采用虚拟存储机制的系统中，每个程序的可执行目标文件在装入时都被映射到同样的虚拟地址空间上，也即，所有用户进程的虚拟地址空间是一致的，只是在相应的只读区域和可读写数据区域中映射的信息不同而已，它们分别映射到对应可执行目标文件中的只读段（节 .init、.text 和 .rodata 组成的段）和可读写数据段（节 .data 和 .bss 组成的段）。其中，只有 .bss 节在可执行目标文件中没有具体内容，因此，在运行时由操作系统将该节对应的存储区初始化为0。

对于 IA-32，其内核虚拟存储空间在 0xc0000000 以上的高端地址上，用来映射到操作系统内核代码和数据、物理存

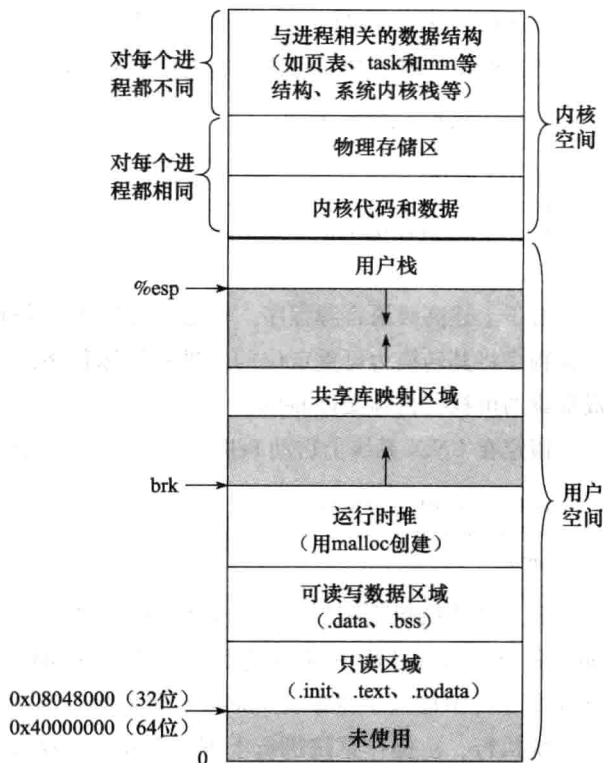


图 7.4 进程的存储映像

储区，以及与每个进程相关的系统级上下文数据结构（如进程标识信息、进程现场信息、页表等进程控制信息以及内核栈等），其中内核代码和数据区在每个进程的地址空间中都相同。用户程序没有权限访问内核空间。

用户空间用来映射到用户进程的代码、数据、堆和栈等用户级上下文信息。每个区域都有相应的起始位置，堆区和栈区相向生长。对于 IA-32，其用户栈区从内核起始位置 0xc0000000 开始向低地址增长，堆栈区中的共享库映射区域从 0x40000000 开始向高地址增长，只读区域从 0x08048000 开始向高地址增长，只读区域后面跟着可读写数据区域，其起始地址通常要求按 4KB 对齐。对于 x86-64，其最开始的只读区域从 0x40000000 开始。

小贴士

Linux 将用户空间对应的进程虚拟存储空间组织成若干“区域（area）”的集合，这些区域是指在虚拟存储空间中的连续片，并且是“已分配页”，例如，只读代码区、可读写数据区、已分配堆区、已分配栈区、共享库映射区等。每个区域可被划分成大小相等的页面。

内核为每个进程维护了一个进程描述符（数据类型为 task_struct 结构，可简称为 task 结构），其中记录或者指向内核运行该进程所需要的所有信息，例如，标识进程的 PID、指向用户栈的指针、可执行目标文件的文件名、程序计数值 PC 等。

task 结构中有个指针指向一个 mm 结构。mm 结构描述了对应进程虚拟存储空间的前状态,其中,有一个字段是 pgd,它指向对应进程的第一级页表(页目录表)的首地址,因此,当处理器运行对应的进程时,内核会将它传送到 CR3 控制寄存器中。mm 结构中还有一个字段 mmap,它指向一个由 vm_area 结构构成的链表表头,每个 vm_area 结构描述了对应进程虚拟存储空间中的一个区域,其中包括以下字段:

- vm_start: 指向区域的开始处。
- vm_end: 指向区域的结束处。
- vm_prot: 描述区域包含的所有页面的访问权限。
- vm_flags: 描述区域包含的页面是否与其他进程共享等。
- vm_next: 指向链表下一个 vm_area 结构。

Linux 采用链表方式管理用户空间中的区域,使得内核不用记录那些不存在的页面(如图 7.4 中灰色区),并且这种页面不占用主存、磁盘或内核本身任何额外资源。

综上所述, Linux 通过 task 结构(包括它所指向的 mm 结构以及 mm 结构所指向的其他结构)记录了一个用户进程(任务)的进程标识信息、进程现场信息和进程控制信息。

7.1.5 程序的加载和运行

在第 4 章 4.5 节介绍可执行目标文件的加载时提到,当启动一个可执行目标文件执行时,首先会通过某种方式调出常驻内存的一个称为加载器(loader)的操作系统程序来进行处理。在 UNIX/Linux 系统中,可以通过调用 execve() 函数来启动加载器。

execve() 函数的功能是在当前进程的上下文中加载并运行一个新程序。execve() 函数的用法如下:

```
int execve(char *filename, char *argv[], *envp[]);
```

该函数用来加载并运行可执行目标文件 filename,可带参数列表 argv 和环境变量列表 envp。若出现错误,如找不到指定文件 filename,则返回 -1 并将控制权返回给调用程序;若函数执行成功,则不返回,最终将控制权传递到可执行目标中的主函数 main。

通常,主函数 main() 的原型形式如下:

```
int main(int argc, char **argv, char **envp);
```

或者是如下的等价形式:

```
int main(int argc, char *argv[], char *envp[]);
```

其中,参数列表 argv 可用一个以 null 结尾的指针数组表示,每个数组元素都指向一个用字符串表示的参数。通常,argv[0] 指向可执行目标文件名,argv[1] 是命令(以可执行目标文件名作为命令的名字)第一个参数的指针,argv[2] 是命令第二个参数的指针,以此类推。参数个数由 argc 指定。参数列表结构如图 7.5 所示。图中显示了命令行“ld -o test main.o test.o”对应的参数列表结构。

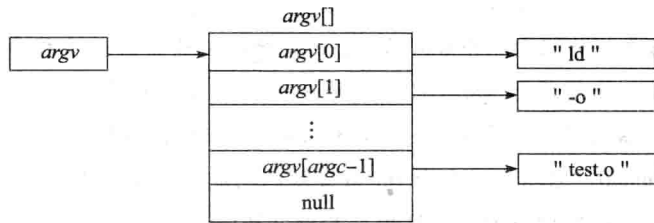


图 7.5 参数列表 *argv* 的组织结构

环境变量列表 *envp* 的结构与参数列表结构类似。也用一个以 *null* 结尾的指针数组表示，每个数组元素都指向一个用字符串表示的环境变量串。其中每个字符串都是一个形如 “NAME = VALUE” 的名 - 值对。

当 IA-32/Linux 系统开始执行 *main()* 函数时，在虚拟地址空间的用户栈中具有如图 7.6 所示的组织结构。

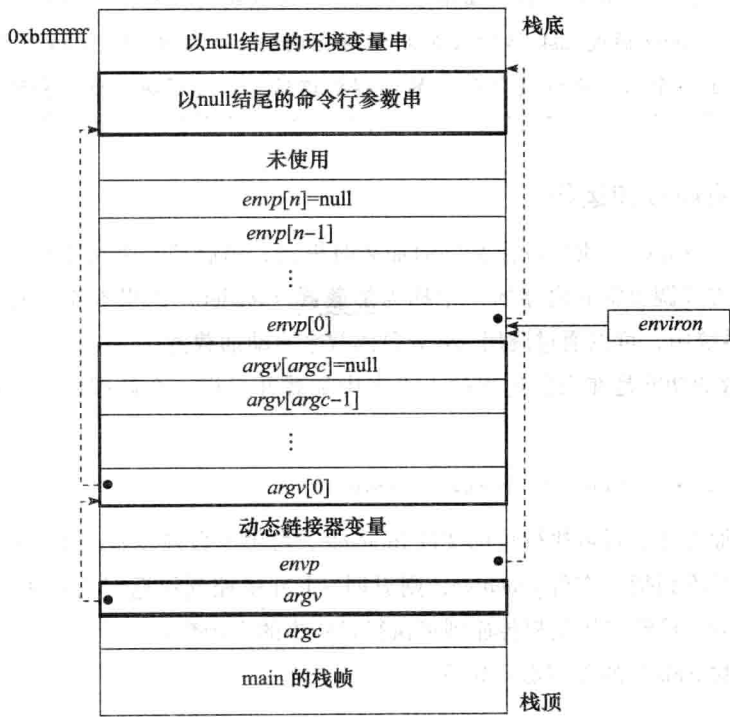


图 7.6 运行一个新程序的 *main* 函数时用户栈中的典型结构

如图 7.6 所示，用户栈的栈底是一系列环境变量串，然后是命令行参数串，每个串以 *null* 结尾，连续存放在栈中，每个串 *i* 由相应的 *envp[i]* 和 *argv[i]* 中的指针指示。在命令行参数串后面是指针数组 *envp* 的数组元素，全局变量 *environ* 指向这些指针中的第一个指针 *envp[0]*。然后是指针数组 *argv* 的数组元素。在栈的顶部是 *main()* 函数的三个参数：*envp*、*argv* 和 *argc*。在这三个参数所在单元的后面将生成 *main()* 函数的栈帧。

对于第 1 章的 1.4.2 节中提到的 *hello* 程序的加载执行，大致过程如下：

- ① shell 命令行解释器输出一个命令行提示符（如：unix >），并开始接受用户输入的命令行。
- ② 当用户在命令行提示符后输入命令行“./hello[Enter]”后，开始对命令行进行解析，获得各个命令行参数并构造传递给函数 `execve()` 的参数列表 `argv`，将参数个数送 `argc`。
- ③ 调用函数 `fork()`，创建一个子进程，新创建的子进程获得与父进程完全相同的虚拟存储空间中的一个备份，包括只读段、可读写数据段、堆以及用户栈等。
- ④ 以第②步命令行解析得到的参数个数 `argc`、参数列表 `argv` 以及全局变量 `environ` 作为参数，调用函数 `execve()`，从而实现在当前进程（新创建的子进程）的上下文中加载并运行 `hello` 程序。在函数 `execve()` 中，通过启动加载器执行加载任务，将可执行目标文件 `hello` 中的 `.text`、`.data`、`.bss` 节等内容加载到当前进程的虚拟地址空间（实际上并没有将 `hello` 文件中的代码和数据从磁盘读入主存，而是修改了当前进程上下文中关于存储映像的一些数据结构）。当加载器执行完加载任务后，便开始转到 `hello` 程序的第一条指令执行，从此，`hello` 程序开始在一个进程的上下文中运行。

7.2 异常和中断

一个进程在正常执行过程中，其逻辑控制流会因为各种特殊事件被打断，例如，上述 7.1 节中提到的操作系统进行的时间片轮转处理器调度，在每个时间片到时，当前进程的执行被新进程打断。除此之外，打断进程正常执行的特殊事件还有：用户按下 Ctrl + C 键、当前指令执行时发生了不能使指令继续执行的意外事件、I/O 设备完成了系统交给的任务需要系统进一步处理等。这些特殊事件统称为异常（exception）或中断（interrupt）。

当发生异常或中断时，正在执行进程的逻辑控制流被打断，CPU 转到具体的处理特殊事件的内核程序去执行。显然，这与上一节介绍的上下文切换一样，都会引起一个异常控制流。但是，它与上下文切换有一个明显的不同：上下文切换后 CPU 执行另一个用户进程，但是，中断或异常处理程序执行的代码不是一个进程，而是一个“内核控制路径”，它代表异常或中断发生时正在运行的当前进程在内核态执行一个独立的指令序列。作为一个内核控制路径，它比进程更“轻”，其上下文信息比一个进程的上下文信息少得多。

不同计算机体系结构和教科书对异常和中断这两个概念规定了不同的内涵。例如，PowerPC 体系结构用“异常”表示各种来自 CPU 内部和外部的意外事件，而用“中断”表示正常程序执行控制流被打断这个概念。在 Randal E. Bryant 等编著的《Computer System: A Programmer's Perspective》中，用“异常”表示所有来自 CPU 内部和外部的意外事件的总称，同时“异常”也表示程序正常执行控制流被打断这个概念，本教材主要以 IA-32 为教学模型机，将使用 Intel 体系结构规定的“中断”和“异常”的概念。

7.2.1 基本概念

在早期的 Intel 8086/8088 微处理器中，并不区分异常和中断，两者统称为中断，由 CPU 内部产生的意外事件称为“内中断”，通过 CPU 的中断请求引脚 INTR 和 NMI 从 CPU 外部发出的中断请求为“外中断”。

从 80286 开始, Intel 统一把“内中断”称为异常, 而把“外中断”称为中断。在 IA-32 架构说明文档中, Intel 对异常和中断进行了如下描述: 处理器提供了异常和中断这两种打断程序正常执行的机制。中断是一种典型地由 I/O 设备触发的、与当前正在执行的指令无关的异步事件; 而异常是处理器执行一条指令时, 由处理器在其内部检测到的、与正在执行的指令相关的同步事件。

有时为了强调异常是 CPU 内部执行指令时发生, 而中断是 CPU 外部的 I/O 设备向 CPU 发出的请求, 特称异常为“内部异常”, 而称中断为“外部中断”。中断是外设的一种输入输出方式, 有关中断的主要内容并不在本章中介绍, 相关内容请参考第 8 章或其他有关教材。

实际上, 异常和中断两者的处理过程和实现机制基本上是相同的, 这也是为什么在有些体系结构或教科书中将两者统称为“中断”或统称为“异常”的原因。

异常和中断引起的异常控制流如图 7.7 所示。图中反映了从 CPU 检测到用户进程发生异常或中断事件, 到 CPU 改变指令执行控制流而转到操作系统中的异常或中断处理程序执行, 再到从异常或中断处理程序返回用户进程执行的过程。

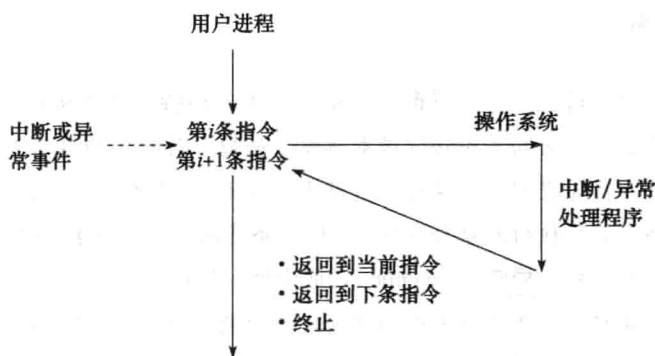


图 7.7 中断和异常处理过程

异常和中断处理的大致过程如下: 当 CPU 在执行当前程序或任务 (即用户进程) 的第 i 条指令时检测到一个异常事件, 或在执行第 i 条指令后发现有一个中断请求信号, 则 CPU 会打断当前用户进程, 然后转到相应的异常或中断处理程序去执行。若异常或中断处理程序能够解决相应问题, 则在异常或中断处理程序的最后, CPU 通过执行“异常/中断返回指令”回到被打断的用户进程的第 i 条指令或第 $i+1$ 条指令继续执行; 若异常或中断处理程序发现是不可恢复的致命错误, 则终止用户进程。通常情况下, 对于异常和中断事件的具体处理过程全部由操作系统 (可能包括驱动程序) 软件来完成。

通常, 把处理异常事件的程序称为异常处理程序, 把处理中断事件的程序称为中断服务程序, 合在一起时本教材称其为异常或中断处理程序。

7.2.2 异常的分类

Intel 将内部异常分为三类: 故障 (fault)、陷阱 (trap) 和终止 (abort)。

1. 故障

故障是在引起故障的指令被启动后但未执行结束时 CPU 检测到的一类与指令执行相关的

意外事件。这种意外事件有些可以恢复，有些则不能恢复。例如，指令译码时出现“非法操作码”；取指令或数据时发生“页故障（page fault）”；执行除法指令时发现“除数为0”等。

对于像溢出和非法操作码等这类故障，因为无法通过异常处理程序恢复，因此不能回到被中断的程序继续执行，通常异常处理程序在屏幕上显示一个对话框告知发生了某种故障，然后调用内核中的 abort 例程，以终止发生故障的当前进程。

对于除数为0的情况，根据是定点除法指令还是浮点除法指令有不同的处理方式。对于浮点数除0，异常处理程序可以选择将指令执行结果用特殊的值（如 ∞ 或NaN）表示，然后返回到用户进程继续执行除法指令后面的一条指令；而对于整数除0，则会发生“整除0”故障，通常调用 abort 例程来终止当前用户进程。

对于页故障，对应的页故障处理程序会根据不同的情况进行不同的处理。根据第6章有关内容可知，CPU在执行指令过程中需要访问存储器时，首先进行地址转换，在查页表进行地址转换时，判断相应页表项中的有效位是否为1，并且确定是否地址越界或访问越权，如果检测到有效位不为1或者地址越界或者访问越权，都会产生“page fault”异常，从而调出内核中相应的异常处理程序执行。因此，CPU产生的“page fault”异常中包含了多种不同情况，需要页故障处理程序根据具体情况进行不同处理：首先检测是否发生地址越界或访问越权，如果是的话，则故障不可恢复；否则是真正的缺页故障，此时，可以通过从磁盘读入页面来恢复故障。Linux中，不可恢复的访存故障（地址越界和访问越权）都称为“段故障（segmentation fault）”。

例 7.1 假设在 IA-32/Linux 系统中一个 C 语言源程序 P 如下：

```
1 int a[1000];
2 int x;
3 main()
4 {
5     a[10] = 1;
6     a[1000] = 3;
7     a[10000] = 4;
8 }
```

假设经过编译、汇编和链接以后，第5、6和7行源代码对应的指令序列如下：

```
5 8048300: c7 05 28 90 04 08 01 00 00 00 movl $0x1, 0x8049028
6 8048309: c7 05 a0 9f 04 08 03 00 00 00 movl $0x3, 0x8049fa0
7 8048313: c7 05 40 2c 05 08 04 00 00 00 movl $0x4, 0x8052c40
```

已知系统采用分页虚拟存储管理方式，页大小为4KB。若在运行P对应的进程时，系统中没有其他进程在运行，则对于上述三条指令的执行，在取指令时是否可能发生页故障？在数据访问时分别会发生什么问题？哪些问题是可恢复的？哪些问题是不可恢复的？

解 对于上述三条指令的执行，访问指令时都不会发生缺页，因为在执行这些指令之前，一定执行过其他位于这些指令前面的指令，它们都位于起始地址为0x08048000（是一个4KB页面的起始位置）的同一个页面，所以，在执行这三条指令之前，它们已经随着前面某条指令一起被装入了内存。因为没有其他进程在系统中运行，所以不会因为执行其他进程而使得调入主存的页面被调出到磁盘。综上所述，执行到这三条指令时，都不会在取指令时发生页故障。

对于第5行对应指令的执行，数据访问时会发生缺页异常，但是是可恢复的故障。因为，对

于地址为 0x8049028 的 $a[10]$ 的访问, 是对所在页面 (起始地址为 0x08049000, 是一个 4KB 页面的起始位置) 的第一次访问, 因而对应页面不在主存。当 CPU 执行到该指令时, 检测到缺页异常, 即发生了页故障, 此时, CPU 暂停用户进程 P 的执行, 将控制转移到操作系统内核, 调出内核中的“页故障处理程序”执行。在页故障处理程序中, 检查是否地址越界或访问越权, 显然这里没有发生越界和越权情况, 故将地址 0x8049028 所在页面从磁盘调入内存, 处理结束后, 再回到这条 `movl` 指令重新执行, 此时, 再访问数据就没有问题了。处理过程如图 7.8 所示。

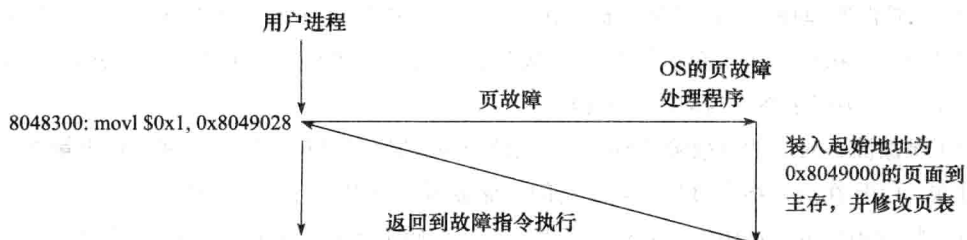


图 7.8 第 5 行指令执行时页故障处理过程

对于第 6 行对应指令的执行, 数据访问时不会发生缺页, 因为在执行上一条指令时, 已经将起始地址为 0x8049000 的页面装入了内存, 而地址 0x8049fa0 位于该页面中 (因为 $4 \times 1000 + 4 = 4004 < 4K$), 因而不会缺页。但是, 因为数组 a 只有 1000 个元素, 即 $a[0] \sim a[999]$, $a[1000]$ 并不存在, 不过, C 编译器可能不会检查数组边界, 因而生成了第 6 行对应的指令 “`movl $0x3, 0x8049fa0`”, 其中的地址 0x8049fa0 可能是 x 的地址而不是 $a[1000]$ 的地址, 即在该指令执行前, 地址 0x8049fa0 中可能存放的是 0 (对 x 初始化为 0), 该指令执行后, 不知不觉地将地址 0x8049fa0 中原来的 0 换成了 3。

对于第 7 行对应指令的执行, 数据访问时很可能会发生页故障, 而且是不可恢复的故障。显然, $a[10000]$ 并不存在, 不过, C 编译器可能会生成第 7 行对应的指令 “`movl $0x4, 0x8052c40`”, 其中的地址 0x8052c40 偏离数组首地址 0x8049000 已达 $4 \times 10000 + 4 = 40004$ 个单元, 即偏离了 9 个页面, 很可能超出了可读写数据区范围, 因而当 CPU 执行该指令时, 很可能发生地址越界或访问越权。若是这样的话, CPU 就通过异常响应机制转到操作系统内核, 即调出内核中的页故障异常处理程序执行。在页故障处理程序中, 检测到发生了地址越界或访问越权, 因而页故障处理程序发送一个“段错误”信号 (SIGSEGV) 给用户进程, 用户进程接收到该信号后就调出一个信号处理程序执行, 该信号处理程序根据信号类型, 在屏幕上显示“段故障 (segmentation fault)”信息, 并终止用户进程。处理过程如图 7.9 所示。

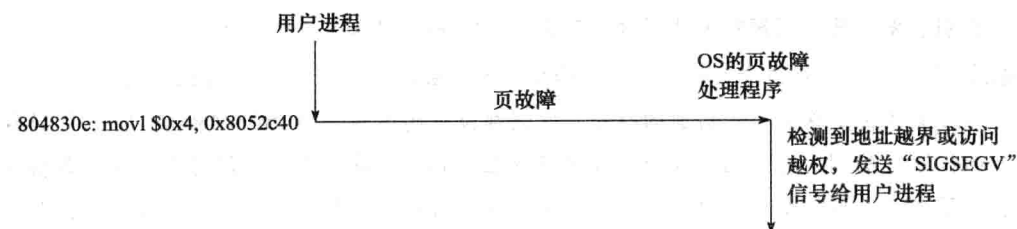


图 7.9 第 7 行指令执行时页故障处理过程

2. 陷阱

陷阱也称为自陷或陷入，与“故障”等其他异常事件不同，是预先安排的一种“异常”事件，就像预先设定的“陷阱”一样。当执行到陷阱指令（也称为自陷指令）时，CPU 就调出特定的程序进行相应的处理，处理结束后返回到陷阱指令的下一条指令执行。其处理过程如图 7.10 所示。

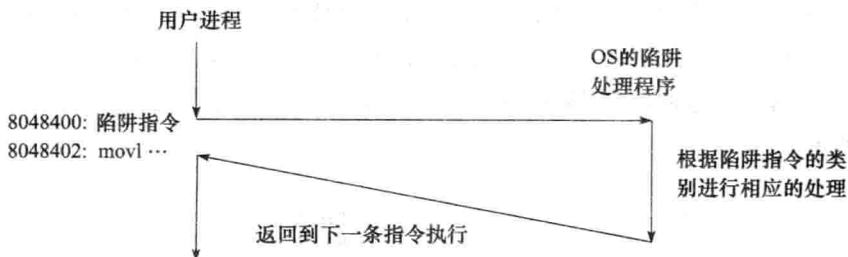


图 7.10 陷阱指令执行时的处理过程

陷阱的重要作用之一是在用户程序和内核之间提供一个像过程一样的接口，这个接口称为系统调用，用户程序利用这个接口可以方便地使用操作系统内核提供的一些服务。操作系统给每个服务编一个号，称为系统调用号，每个服务功能通过一个对应的系统调用服务例程提供。例如，在 Linux 系统中就提供了创建子进程（fork）、读文件（read）、加载并运行新程序（execve）等服务功能，系统调用 fork、read 和 execve 的调用号分别是 1、3 和 11。

为了使用户程序在需要调用内核服务功能的时候，能够从用户态转到对应的系统调用执行，处理器会提供一个或多个特殊的系统调用指令，如 IA-32 处理器中的 sysenter 指令、MIPS 处理器中的 syscall 指令等。这些系统调用指令属于陷阱指令，当执行到这些指令时，CPU 通过一系列步骤自动调出内核中对应的系统调用服务例程执行。

此外，利用陷阱机制可以实现程序调试功能，包括设置断点和单步跟踪。

例如，在 IA-32 中，当 CPU 处于单步跟踪状态（TF = 1 且 IF = 1）时，每条指令都被设置成了陷阱指令，执行每条指令后，都会发生中断类型为 1 的“调试”异常，从而转去执行特定的“单步跟踪处理程序”。该程序将当前指令执行的结果显示在屏幕上。单步跟踪处理过程中，CPU 会自动把标志寄存器压栈，然后将 TF 和 IF 清 0，这样，在单步跟踪处理程序执行过程中，CPU 能以正常方式工作。单步处理结束、返回到断点处执行之前，再从栈中取出标志，以恢复 TF 和 IF 的值，使 CPU 回到单步跟踪状态，这样，下条指令又是“陷阱”指令，将被跟踪执行。如此下去，每条指令都被跟踪执行，直到将 TF 或 IF 清 0 为止。注意，对于“单步跟踪”这类陷阱，当陷阱指令是转移指令时，处理后不能返回到转移指令的下条指令执行，而是返回到转移目标指令执行。

在 IA-32 中，用于程序调试的“断点设置”陷阱指令为 int 3，对应机器码为 CCH，若调试程序在被调试程序某处设置了断点，则调试程序就在该处加入一条 int 3 指令。当 CPU 执行到该指令时，就会暂停当前被调试程序的运行，并发出一个“EXCEPTION_BREAKPOINT”异常，从而最终调出相应的调试程序来执行，执行结束后再回到被设定断点的被调试程序执行。

在 IA-32 中，陷阱指令引起的异常称为编程异常（programmed exception），这些指令包括

INT n 、int 3、into（溢出检查）、bound（地址越界检查）等。通常将 INT n 称为软中断指令，执行该指令引起的“异常”通常也称为软中断（software interrupt）。在 IA-32/Linux 系统中，可以使用快速系统调用指令 sysenter 或者软中断指令 int \$0x80（即 INT n 指令中 $n = 128$ 时）进行系统调用。

3. 终止

如果在执行指令过程中发生了严重错误，例如，控制器出现问题，访问 DRAM 或 SRAM 时发生校验错等，则程序将无法继续执行，只好终止发生问题的进程，在有些严重的情况下，甚至要重启系统。显然，这种异常是随机发生的，无法确定发生异常的是哪条指令。其处理过程如图 7.11 所示。

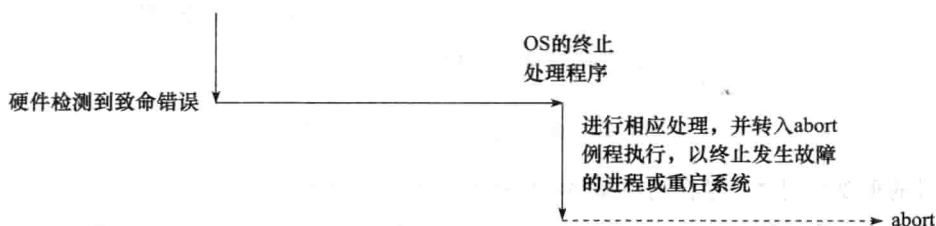


图 7.11 终止异常执行时的处理过程

7.2.3 中断的分类

中断是由外部 I/O 设备请求处理器进行处理的一种信号，它不是由当前执行的指令引起的。外部 I/O 设备通过特定的中断请求信号线向 CPU 提出中断申请。CPU 在执行指令的过程中，每执行完一条指令就查看中断请求引脚，如果中断请求引脚的信号有效，则进入中断响应周期。在中断响应周期中，CPU 先将当前 PC 值（称为断点）和当前的机器状态保存到栈中，并设置成“关中断”状态，然后，从数据总线读取中断类型号，根据中断类型号跳转到对应的中断服务程序执行。中断响应过程由硬件完成，而中断服务程序执行具体的中断处理工作，中断处理完成后，再回到被打断程序的“断点”处继续执行。中断的整个处理过程如图 7.12 所示。

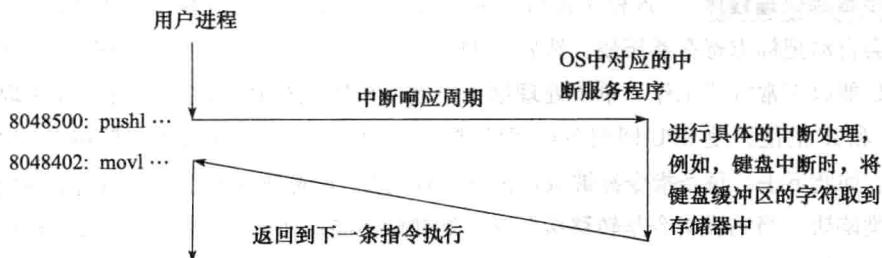


图 7.12 外部中断的处理过程

Intel 将外部中断分成可屏蔽中断（maskable interrupt）和不可屏蔽中断（nonmaskable interrupt, NMI）。

1. 可屏蔽中断

可屏蔽中断是指通过可屏蔽中断请求线 INTR 向 CPU 进行请求的中断，主要来自 I/O 设备

的中断请求。CPU 可以通过在中断控制器中设置相应的屏蔽字来屏蔽它或不屏蔽它，若一个 I/O 设备的中断请求被屏蔽，则它的中断请求信号将不会被送到 CPU。

2. 不可屏蔽中断

不可屏蔽中断通常是非常紧急的硬件故障，通过专门的不可屏蔽中断请求线 NMI 向 CPU 发出中断请求。如：电源掉电，硬件线路故障等，这类中断请求信号一旦产生，任何情况下它都不可以被屏蔽，因此一定会被送到 CPU，以便让 CPU 快速处理这类紧急事件。通常，这种情况下，中断服务程序会尽快保存系统重要信息，然后在屏幕上显示相应的消息或直接重启系统。

7.2.4 异常和中断的响应过程

每种处理器架构都会各自定义它所处理的异常和中断类型，而且，对于异常和中断的处理，不同的处理器/操作系统平台可能也有所不同，但是，它们之间的差别不大，基本原理相同。

在 CPU 执行指令过程中，如果发生了异常事件或中断请求，则 CPU 必须进行相应的处理。CPU 从检测到异常或中断事件，到调出相应的异常或中断处理程序开始执行，整个过程称为“异常和中断的响应”。CPU 对异常和中断的响应过程可分为以下三个步骤：保护断点和程序状态、关中断、识别异常和中断事件并转相应处理程序执行。

1. 保护断点和程序状态

前面提到，对于不同的异常事件，其返回地址（即断点）不同。例如，“缺页故障”异常的断点是发生页故障的当前指令的地址；“陷阱”异常的断点则是陷阱指令后面一条指令的地址。显然，断点与异常类型有关。为了能在异常处理后正确返回到原被中断的程序继续执行，数据通路必须能正确计算断点处的地址。假定计算出的断点地址存放在 PC 中，则保护断点时，只要将 PC 送到堆栈或特定的寄存器中即可。

为了能够支持异常或中断的嵌套处理，大多数处理器将断点保存在栈中，如 IA-32 处理器的断点被保存在栈中；如果系统不支持嵌套处理，则可以将断点保存在特定寄存器中，而不需送到栈中保存，如 MIPS 处理器用 EPC 寄存器专门存放断点。显然，后者 CPU 用于中断的开销较小，因为访问栈就是访问存储器，它比访问寄存器所用的时间要长。

因为异常处理后可能还要回到原被中断的程序继续执行，所以，被中断时原程序的状态（如产生的各种标志信息、允许自陷标志等）都必须保存起来。通常每个正在运行程序的状态信息存放在一个专门的寄存器中，这些专门寄存器统称为程序状态字寄存器（PSWR），存放在 PSWR 中的信息称为程序状态字（Program Status Word，简称 PSW）。例如，在 IA-32 中，程序状态字寄存器就是标志寄存器 EFLAGS。与断点一样，PSW 也要被保存到栈或特定寄存器中，在异常返回时，将保存的 PSW 恢复到 PSWR 中。

2. 关中断

假定在执行异常或中断处理程序时又发生了新的异常或中断，怎么办？如果异常或中断处理程序在保存原被打断程序现场的过程中又发生了新的异常或中断，那么，就会因为要响应和处理新的异常或中断，进而把原被打断程序的现场以及已保存的断点和程序状态等破坏掉，因

此,应该有一种机制来禁止在处理异常或中断时再响应新的异常或中断。通常通过设置“中断允许位”(“中断允许”触发器)来实现。当中断允许位被置1,则为开中断,表示允许响应中断或异常;若中断允许位被清0,则为关中断,表示不允许响应中断或异常。例如,IA-32中的“中断允许位”就是EFLAG寄存器中的中断标志位IF。

在异常或中断响应过程中,通常由CPU将中断允许位清0,以进行关中断操作。例如,对于IA-32/Linux平台,CPU在异常或中断响应过程中,大部分情况下(除into、bound和int \$0x80指令引起的异常外)都会将标志寄存器EFLAGS中的IF清0,以禁止响应新的可屏蔽中断和异常。

在IA-32中,CPU也可以在异常或中断处理程序中,通过执行指令sti或cli,将标志寄存器EFLAGS中的IF位置1或清0,以使CPU处在开中断或关中断状态。

3. 识别异常和中断事件并转相应的处理程序执行

在调出异常和中断处理程序之前,必须知道发生了什么异常或哪个I/O设备发出了中断请求。一般来说,内部异常事件和外部中断源的识别方式不同,大多数处理器会将两者分开来处理。

内部异常事件的识别很简单。只要CPU在执行指令时把检测到的事件对应的异常类型号或标识异常类型的信息记录到特定的内部寄存器中即可。

外部中断源的识别比较复杂。由于外部中断的发生与CPU正在执行的指令没有必然联系,相对于指令来说,外部中断是随机的,与当前执行指令无关,因此并不能根据指令执行过程中的某些现象来判断是否发生了“中断”请求。对于外部中断,只能在每条指令执行完后、取下条指令之前去查询是否有中断请求。通常CPU通过采样对应的中断请求引脚(如Intel处理器的INTR)来进行查询。如果发现中断请求引脚有效,则说明有中断请求,但是,到底是哪个I/O设备发出的请求,还需要进一步识别。通常是由CPU外部的中断控制器根据I/O设备的中断请求和中断屏蔽情况,结合中断响应优先级,来识别当前请求的中断类型号,并通过数据总线将中断类型号送到CPU。有关中断响应处理的详细内容参见8.3.4节中的相关部分。

异常和中断源的识别可以采用软件识别或硬件识别两种方式。

软件识别方式是指,CPU中设置一个原因寄存器,该寄存器中有一些标识异常原因或中断类型的标志信息。操作系统使用一个统一的异常或中断查询程序,该程序按一定的优先级顺序查询原因寄存器,先查询到的先被处理。例如,MIPS就采用软件识别方式,CPU中有一个cause寄存器,位于地址0x80000180处有一个专门的异常/中断查询程序,它通过查询cause寄存器来检测异常和中断类型,然后转到内核中相应的异常处理程序或中断服务程序进行具体的处理。

硬件识别方式称为向量中断方式。这种方式下,异常或中断处理程序的首地址称为中断向量,所有中断向量存放在一个表中,称为中断向量表。每个异常和中断都被设定一个中断类型号,中断向量存放的位置与对应的中断类型号相关,例如,类型0对应的中断向量存放在第0表项,类型1对应的中断向量存放在第1表项,以此类推,因而可以根据类型号快速找到对应的处理程序。IA-32中的异常和中断识别就是采用这种向量中断方式。

* 7.2.5 IA-32 的中断向量表

IA-32 采用向量中断方式, 可以处理 256 种不同类型的异常和中断, 每个异常或中断都有唯一的编号, 称之为中断类型号 (也称向量号), 并且还有与其对应的异常处理程序或中断服务程序, 其入口地址放在一个专门的中断向量表中。例如, 类型 0 为“除法错”, 类型 2 为“NMI 中断”, 类型 14 为“缺页”等。

IA-32 中前 32 个中断类型 (0 ~ 31) 保留给处理器使用, 剩余的可以由用户自行定义功能, 这里的用户是指机器硬件的用户, 实际上就是操作系统。通过执行指令 $\text{INT } n$ (指令的第二字节给出中断类型号 n , $n = 32 \sim 255$) 使 CPU 自动转到处理器的用户 (即操作系统) 编写的中断服务程序执行。

在实地址模式下, 异常处理程序或中断服务程序的入口地址 (由 16 位段地址和 16 位偏移地址组成) 称为中断向量, 用来存放 256 个中断向量的数据结构就是中断向量表。由于在实地址模式下每个中断向量占 4 个字节, 其中高 16 位是段地址, 低 16 位是偏移地址, 因此 256 个中断向量组成的中断向量表需要 $256 \times 4\text{B} = 1\text{KB}$ 内存空间, 并固定在 $00000\text{H} \sim 003\text{FFH}$ 的内存区域内。

小贴士

实地址模式 (Real Mode) 是 Intel 为 80286 及其之后的处理器提供的一种8086 兼容模式。采用 20 位存储器地址空间, 即可寻址空间为 1MB, 不支持分页存储管理机制。每个存储单元地址由 16 位段地址左移 4 位后与 16 位偏移量相加而得到。

开机后系统首先在实地址模式下工作, 因此, 开机过程中, 需要先准备在实地址模式下的中断向量表和中断服务程序。通常, 这个准备工作是由固化在计算机主板上的一块 ROM 芯片中的BIOS 程序来完成的。BIOS 程序首先检测显卡、键盘、内存等, 并在主存的 $00000\text{H} \sim 003\text{FFH}$ 区域建立中断向量表, 同时, 在中断向量所指的主存区域建立相应的中断服务程序。利用这些中断服务程序可以把操作系统内核程序从磁盘加载到内存中。例如, BIOS 可以通过执行指令 $\text{int } 0\text{x}19$ 来调用中断向量 $0\text{x}19$ 对应的中断服务程序, 将启动盘上的 0 号磁头对应盘面的 0 磁道 1 扇区中的引导程序装入内存。

BIOS (Basic Input/Output System) 是基本输入/输出系统的简称, 是针对具体的主板设计的, 与安装的操作系统无关。BIOS 中包含了各种基本设备的驱动程序, 通过执行 BIOS 程序, 这些基本设备驱动程序以中断服务程序的形式被加载到内存中, 以提供基本 I/O 系统调用。一旦进入保护模式, 就不再使用 BIOS。

* 7.2.6 IA-32 的中断描述符表

IA-32 的保护模式并不像实地址模式那样将异常处理程序或中断服务程序的入口地址直接填入 $00000\text{H} \sim 003\text{FFH}$ 存储区, 而是借助于中断描述符表来获得异常处理程序或中断服务程序的入口地址。

表 7.1 给出了 IA-32 中常见的中断和异常类型, 表中内容包括类型 (向量) 号、助记符、含义、事件源。

表 7.1 IA-32 的异常/中断类型

类型号	助记符	含义描述	起因或事件源
0	#DE	除法出错	div 和 idiv 指令
1	#DB	单步跟踪	任何指令和数据引用
2		NMI 中断	不可屏蔽外部中断
3	#BP	断点	int 3 指令
4	#OF	溢出	into 指令
5	#BR	边界检测 (BOUND)	bound 指令
6	#UD	无效操作码	不存在的指令操作码
7	#NM	协处理器不存在	浮点或 wait/fwait 指令
8	#DF	双重故障	处理一个异常时发生另一个
9	#MF	协处理器段越界	浮点指令
10	#TS	无效 TSS	任务切换或访问 TSS
11	#NP	段不存在	需装入段寄存器或访问系统段
12	#SS	栈段错	栈操作和装入 SS 段寄存器
13	#GP	一般性保护错 (GPF)	存储器引用和其他保护检查
14	#PF	页故障	存储器引用
15		保留	
16	#MF	浮点错误	浮点或 wait/fwait 指令
17	#AC	对齐检测	存储器数据引用
18	#MC	机器检测异常	与机器具体型号有关
19	#XM	SIMD 浮点异常	SIMD 浮点指令
20 ~ 31		保留	
32 ~ 255		可屏蔽中断和软中断	INTR 中断或 INT n 指令

IA-32 中定义了 19 种确定的中断或异常类型，类型号为 0 ~ 14 和 16 ~ 19。在 19 种预定义的类型中，大部分都是故障类异常，主要是由于执行了特定的指令而引起的。例如，在执行整数除法指令 div 和 idiv 时，若发现除数为 0 或结果溢出，则发生 0 号异常，即除法错故障 (#DE)。在执行每条指令时，若发现 TF = 1 且 IF = 1，则发生 1 号异常，即单步跟踪调试陷阱 (#DB)。在执行 int 3 指令时，则发生 3 号异常，即断点陷阱 (#BP)。在执行 into 指令时，若 OF = 1，则发生 4 号异常，即溢出故障 (#OF)；在执行 bound 指令时，若发现数组下标越界，则发生 5 号异常，即范围越界故障 (#BR)。在执行指令过程中，如果在引用存储器以访问指令或数据时发生缺页，则产生 14 号异常，即页故障 (#PF)。

除了 19 种确定的类型外，还有 224 种用户自定义类型，类型号为 32 ~ 255，其中一部分类型用于外部可屏蔽中断，一部分用于软中断。可屏蔽中断通过 CPU 的 INTR 引脚向 CPU 发出中断请求，对应中断请求号为 IRQ0、IRQ1、…、IRQ15 等。软中断指令 INT n 被设定为一种陷阱异常，例如，Linux 通过 int \$0x80 指令将 128 号设定为系统调用，而 Windows 通过 int \$0x2e 指令将 46 号设定为系统调用。

保护模式下，借助于中断描述符表来获得异常处理程序或中断服务程序的入口地址。**中断描述符表** (Interrupt Descriptor Table, IDT) 是操作系统内核中的一个表，共有 256 个表项，对应表 7.1 所示的 256 个异常和中断，因为每个表项占 8 个字节，因而，IDT 共占用 $256 \times 8\text{B} = 2\text{KB}$ 内存空间。

每一个表项是一个中断门描述符、陷阱门描述符或任务门描述符，如图 7.13 所示是中断门描述符格式。

偏移地址 (A31~A16)															
P	DPL	0	1	1	1	0	0	0	0	0	0	0	0	0	0
段选择符															
偏移地址 (A15~A0)															

图 7.13 中断门描述符格式

陷阱门描述符和任务门描述符的格式类似于中断门描述符的，其中，都有一个字段 P 和字段 DPL，其含义与第 6 章 6.6.1 节中介绍的段描述符中规定的一致。P=1 表示段存在，P=0

表示段不存在。Linux 总是把 P 置 1，因为它从来不会把一个段交换到磁盘上，而是以页面为单位交换。DPL 给出访问本段要求的最低特权等级。因此，DPL 为 0 的段只能当 CPL 为 0（内核态）时才可访问，而对于 DPL 为 3 的段，则任何进程都可以访问。在 DPL 后面的一位都是 0，再后面 4 位用来标识门的类型（TYPE）。若是中断门，则 TYPE=1110B；若是陷阱门，则 TYPE=1111B；若是任务门，则 TYPE=0101B。

中断门描述符和陷阱门描述符中都会给出一个 16 位的段选择符和 32 位的偏移地址。段选择符用来指示异常处理程序或中断服务程序所在段的段描述符在 GDT 中的位置，偏移地址则给出异常处理程序或中断服务程序第一条指令所在的偏移量。Linux 利用陷阱门来处理异常，利用中断门来处理中断。因为异常和中断对应的处理程序都属于内核代码段，所以，所有中断门和陷阱门的段选择符都指向 GDT 中的“内核代码段”描述符。有关 Linux 内核代码段描述符的设置可参见第 6 章的 6.6.1 节。通过中断门进入到一个中断服务程序时，CPU 会清除 EFLAGS 寄存器中的 IF 标志，即关中断；而在通过陷阱门进入一个异常处理程序时，CPU 不会修改 IF 标志。也就是说，外部中断不支持嵌套处理，而内部异常则支持嵌套处理。

任务门描述符中不包含偏移地址，只包含 TSS 段选择符，这个段选择符指向 GDT 中的一个 TSS 段描述符，CPU 根据 TSS 段中的相关信息装载 EIP 和 ESP 等寄存器，从而执行相应的异常处理程序。Linux 中，将型号为 8 的双重故障（#DF）用任务门实现，而且它是唯一通过任务门实现的异常。双重故障 TSS 段描述符在 GDT 中位于索引值为 0x1f 的表项处，即 13 位索引为 0 0000 0001 1111，且其 TI=0（指向 GDT），RPL=00（内核级代码），根据图 6.36 的段选择符格式可知，任务门描述符中的段选择符为 00F8H。

* 7.2.7 IA-32 中异常和中断的处理

本节描述 IA-32 处理器如何处理异常和中断，假定操作系统内核已被初始化，因此，以下描述的情形是处理器在保护模式下运行的情况。

在 IA-32 中，每条指令执行后，下条指令的逻辑地址（虚拟地址）由寄存器 CS 和 EIP 指示。在每条指令执行过程中会根据执行情况判定是否发生了某种内部异常事件，在每条指令执行结束时判定是否发生了外部中断，因此，在 CPU 根据 CS 和 EIP 去取下条指令之前，会根据检测的结果判断是否进入异常和中断响应阶段。

若检测到有异常或中断发生，则进入异常和中断响应阶段，在此期间 CPU 的控制逻辑完成以下工作。

- ① 确定检测到的中断类型号 i ，从 IDTR 指向的 IDT 中取出第 i 个表项 $IDTi$ 。
- ② 根据 $IDTi$ 中的段选择符，从 GDTR 指向的 GDT 中取出相应的段描述符，得到对应异常

处理程序或中断服务程序所在段的 DPL、基地址等信息。Linux 下即为内核代码段，所以 DPL 为 0，基地址为 0。

③ 将当前特权级 CPL (CS 寄存器最低两位，00 为内核特权级，11 为用户特权级) 与段描述符中的 DPL 比较。若 CPL 小于 DPL，则产生 13 号异常 (#GP)。Linux 中，内核代码段的 DPL 总是 0，因此不管怎样都不会发生 CPL 小于 DPL 的情况。

对于编程异常，还需进一步做以下检查：若 IDT_i 门描述符中的 DPL 小于 CPL，则产生 13 号异常。这个检查主要是为了防止恶意应用程序通过 INT *n* 指令模拟非法异常和中断以进入内核态执行非法的破坏性操作。

④ 检查是否发生了特权级变化，即判断 CPL 是否与相应段描述符中的 DPL 不同。如果是的话，就需要从用户态切换至内核态，以使用内核对应的栈。通过以下步骤完成栈的切换：

- 读 TR 寄存器，以访问正在运行进程的 TSS 段；
- 将 TSS 段中保存的内核栈的段选择符和栈指针分别装入寄存器 SS 和 ESP，然后在内核栈中保存原来的用户栈的 SS 和 ESP。

Linux 中，若 CPL = DPL，则发生异常或中断的指令也在内核态执行；若 CPL 大于 DPL，则从用户态转到内核态执行，因此，应从用户栈切换到内核栈。

⑤ 如果发生的是故障，则将发生故障的指令的逻辑地址写入 CS 和 EIP，以保证故障处理后能回到发生故障的指令执行。其他情况下，CS 和 EIP 内容不变，使得异常或中断处理后回到下一条指令执行。

⑥ 在当前栈中保存 EFLAGS、CS 和 EIP 寄存器的内容。

⑦ 如果异常产生了一个硬件出错码，则将其保存在内核栈中。

⑧ 将 IDT_i 中的段选择符装入 CS，IDT_i 中的偏移地址装入 EIP，它们是异常处理程序或中断服务程序第一条指令的逻辑地址。

这样，从下一个时钟开始，就执行异常处理程序或中断服务程序的第一条指令。在异常处理程序或中断服务程序中，处理完异常或中断后，通过执行最后一条指令 IRET 回到原被中断的进程继续执行。

CPU 在执行 IRET 指令的过程中完成以下工作。

- ① 从栈中弹出硬件出错码（如果保存过的话）、EIP、CS 和 EFLAGS。
- ② 检查当前异常或中断处理程序的 CPL 是否等于 CS 中最低两位，若是，则说明异常或中断响应前后都处于同一个特权级，IRET 指令完成操作；否则，再继续完成下一步工作。
- ③ 从内核栈中弹出 SS 和 ESP，以恢复到异常或中断响应前的特权级进程所使用的栈。
- ④ 检查 DS、ES、FS 和 GS 段寄存器的内容，若其中有某个寄存器的段选择符指向一个段描述符且其 DPL 小于 CPL，则将该段寄存器清 0。这是为了防止恶意应用程序 (CPL = 3) 利用内核以前使用过的段寄存器 (DPL = 0) 来访问内核地址空间。

显然，执行完 IRET 指令后，CPU 自然回到原来发生异常或中断的进程继续执行。

* 7.2.8 Linux 对异常和中断的处理

异常和中断的处理由 CPU 和操作系统协调完成。CPU 在执行指令过程中检测到异常或中

断事件后,通过对异常和中断的响应,调出异常处理程序或中断服务程序来执行。其中,CPU 负责对异常和中断的检测与响应,而操作系统则负责初始化 IDT 以及编制好异常处理程序或中断服务程序。

1. IDT 的初始化

IA-32 提供了三种包含在 IDT 中的门描述符:中断门、陷阱门和任务门。Linux 运用提供的三种门描述符格式,构造了以下 5 种类型的门描述符。

① 中断门 (interrupt gate): $DPL = 0$, $TYPE = 1110B$ 。所有 Linux 中断服务程序都通过中断门激活。

② 系统门 (system gate): $DPL = 3$, $TYPE = 1111B$ 。Linux 使用系统门激活三个陷阱型异常处理程序,它们的中断类型号是 4、5 和 128,分别对应指令 `into`、`bound` 和 `int $0x80`。因为 DPL 为 3,故任何情况下 $CPL \leq DPL$,所以在用户态下可以使用这三条指令。

③ 系统中断门 (system interrupt gate): $DPL = 3$, $TYPE = 1110B$ 。Linux 使用系统中断门激活 3 号中断(即断点)的异常处理程序,对应指令 `int 3`。因为 DPL 为 3,故任何情况下 $CPL \leq DPL$,所以用户态下可以使用 `int 3` 指令。

④ 陷阱门 (trap gate): $DPL = 0$, $TYPE = 1111B$ 。Linux 用陷阱门阻止用户程序使用 `INT  $n$` ($n \neq 128$ 或 3) 指令模拟非法异常来陷入内核态运行。因为编程异常需要进一步检查门的 DPL 是否小于 CPL ,若是的话,该指令无法通过陷阱门,会出现 13 号异常(`#GP`,通用保护错)。这里将 DPL 设为 0,那么在执行用户程序中的 `INT  $n$` 指令时,因为 $CPL = 3$,故 CPL 大于 DPL ,因而 CPU 会检测到 `#GP` 异常,从而阻止非法 `INT  $n$` 指令的执行。

⑤ 任务门 (task gate): $DPL = 0$, $TYPE = 0101B$ 。Linux 中对 8 号中断(双重故障)用任务门激活。

Linux 内核在启用异常和中断机制之前,需要先设置好每个 IDT 的表项 $IDTi$ ($0 \leq i < 256$),并把 IDT 的首地址存入 IDTR 寄存器。这个工作是在系统初始化时完成的。

系统初始化时, Linux 完成对 GDT、GDTR、IDT 和 IDTR 等的设置,这样,以后一旦发生异常或中断,则 CPU 可以通过异常和中断响应机制调出异常或中断处理程序执行。

Linux 对异常和中断的处理有不同的考虑。

2. 对异常的处理

对于 IA-32 产生的大部分异常, Linux 都解释为一种出错条件。当硬件检测到异常发生后,硬件通过异常响应机制调出对应的异常处理程序,异常处理程序发送一个对应的信号给发生异常的当前进程,或者恢复故障后返回到当前进程的逻辑控制流中的断点处执行。例如,如果某个进程执行了一条带有非法操作码的指令, CPU 就产生一个 6 号异常(`#UD`),在对应的异常处理程序中,向当前进程发送一个 `SIGILL` 信号,以通知当前进程终止运行。

Linux 采用向发生异常的进程发送信号的机制实现异常处理,可以尽快完成在内核态的异常处理过程,因为异常处理过程越长,嵌套执行异常或中断的可能性越大,而异常和中断的嵌套执行会付出较大的代价。

并不是所有异常处理都只是发送一个信号到发生异常的进程。例如,对于 14 号页故障异常(`#PF`),在页故障处理程序中,需要判断是否访问越级(如用户态下访问内核空间)、访问

越权（如修改只读区的信息）或访问越界（如访问了无效存储区）等，如果发生了这些无法恢复的故障，则页故障处理程序发送 SIGSEGV 信号给发生页故障异常的进程；如果没有发生越级、越权和越界错误而只是所需内容不在主存，则页故障处理程序负责把所缺失页面从磁盘装入主存，然后返回到发生缺页故障的指令继续执行。页故障异常情况非常复杂，有关处理细节可以参看操作系统相关教材。

所有异常处理程序的结构是一致的，都可以划分成以下三个部分。

① 准备阶段：在内核栈中保存各寄存器的内容（称为现场信息），这部分大多用汇编语言程序实现。

② 处理阶段：采用 C 函数进行具体的异常处理。执行异常处理的 C 函数名总是由 do_前缀和处理程序名组成，如 do_overflow 函数表示溢出处理程序。其中的大部分函数把硬件出错码和类型号保存在发生异常的当前进程的描述符中，然后向当前进程发送一个对应的信号。异常处理结束时，内核将检查是否发送过某种信号给当前进程。若没有发送，则继续第③步；若发送过信号，则强制当前进程接收信号，异常处理结束。当前进程接收到一个信号后，如果有对应的信号处理程序，则转到信号处理程序执行，执行结束后，返回到当前进程的逻辑控制流的断点处继续执行；如果没有对应的信号处理程序，则调用内核的 abort 例程终止当前进程。表 7.2 给出了每种异常对应的处理程序名以及信号名。

③ 恢复阶段：恢复保存在内核栈中的各个寄存器的内容，切换到用户态并返回到当前进程的逻辑控制流的断点处继续执行。

表 7.2 Linux 中异常对应的信号名和处理程序名

类型号	助记符	含义描述	处理程序名	信号名
0	#DE	除法出错	divide_error()	SIGFPE
1	#DB	单步跟踪	debug()	SIGTRAP
2		NMI 中断	nmi()	无
3	#BP	断点	int3()	SIGTRAP
4	#OF	溢出	overflow()	SIGSEGV
5	#BR	边界检测 (BOUND)	bounds()	SIGSEGV
6	#UD	无效操作码	invalid()	SIGILL
7	#NM	协处理器不存在	device_not_available()	无
8	#DF	双重故障	doublefault()	无
9	#MF	协处理器段越界	coprocessor_segment_overrun()	SIGFPE
10	#TS	无效 TSS	invalid_tss()	SIGSEGV
11	#NP	段不存在	segment_not_present()	SIGBUS
12	#SS	栈段错	stack_segment()	SIGBUS
13	#GP	一般性保护错 (GPF)	general_protecton()	SIGSEGV
14	#PF	页故障	page_fault()	SIGSEGV
15		保留	无	无
16	#MF	浮点错误	coprocessor_error()	SIGFPE
17	#AC	对齐检测	alignment_check()	SIGSEGV
18	#MC	机器检测异常	machine_check()	无
19	#XM	SIMD 浮点异常	simd_coprocessor_error()	SIGFPE

从表 7.2 可以看出,所有与存储访问相关的异常,其对应的信号都是 SIGSEGV,所有与协处理器和浮点运算相关的异常,其对应的信号都是 SIGFPE,在 Linux 中,除法错(除数为 0 或带符号整数除法结果无法表示)也归类为浮点异常。1 号(单步跟踪)和 3 号(断点)的信号都是 SIGTRAP,因而都转到一个专门的用于程序调试的信号处理程序去执行。

3. 对中断的处理

对于大部分异常, Linux 只是给引起异常的当前进程发送一个信号就结束异常处理,这种情况下,具体的异常处理要等到当前进程接收到信号并转到信号处理程序才能进行,而且大部分情况下,异常对应的信号处理结果就是终止当前进程。

显然,这种方式不适合中断的处理。因为中断事件的发生与正在执行的当前进程很可能没有关系,因而将一个信号发给当前进程是没有意义的。

Linux 中处理的中断有以下三种类型。① I/O 中断: 由 I/O 外设发出的中断请求;② 时钟中断: 由某个时钟产生的中断请求,告知一个固定的时间间隔到;③ 处理器中断: 多处理器系统中其他处理器发出的中断请求。后面两种中断的情况超出了本教材的范围。

对于 I/O 中断,每个能够发出中断请求的外部设备控制器都有一条 IRQ 线,所有外设的 IRQ 线都会连接到一个 可编程中断控制器 (Programmable Interrupt Controller, PIC) 中对应的 IRQ 引脚上, PIC 中每个 IRQ 引脚 都有一个编号,如 IRQ0、IRQ1、…、IRQ_i,这里编号 *i* 就是 IRQ 的值,通常将与 IRQ_i 关联的中断的类型号设定为 32 + *i*,传统的方式是将两片 PIC 级联起来,每片有 8 个 IRQ 引脚,两片级联后共有 15 个 IRQ 引脚,因为有一个 PIC 上的 IRQ 引脚需要连到另一片的 INT 引脚上。有关 PIC 的基本结构见第 8 章的图 8.16。

PIC 需要对所有外设发来的 IRQ 请求按照优先级进行排队,如果至少有一个 IRQ 线有请求且未被屏蔽,则 PIC 向 CPU 的 INTR 引脚发中断请求。CPU 每执行完一条指令后都会查询 INTR 引脚,若发现有中断请求,则进入中断响应过程,调出中断服务程序执行。

所有 中断服务程序 的结构类似,都可划分为以下三个阶段。

① 准备阶段: 在内核栈中保存各寄存器的内容(称为现场信息)以及所请求 IRQ 的值等,并给发出中断请求信号的 PIC 回送应答信息,允许其发送新的中断请求信号。因为在响应中断的过程中, CPU 已经禁止了 PIC 发送中断请求的功能。

② 处理阶段: 执行 IRQ 对应的 中断服务例程 (Interrupt Server Routine, ISR)。

③ 恢复阶段: 恢复保存在内核栈中的各个寄存器的内容,切换到用户态并返回到当前进程的逻辑控制流的断点处继续执行。

有关中断处理的详细内容将在 8.3.4 节和 8.4.3 节介绍。

* 7.2.9 IA-32/Linux 的系统调用

系统调用是一种特殊的“异常事件”,是操作系统为用户程序提供服务的一种手段。Linux 提供了几百种系统调用,主要分为以下几类:进程控制、文件操作、文件系统操作、系统控制、内存管理、网络管理、用户管理和进程通信。系统调用号用整数表示,它用来确定系统调用跳转表中的偏移量。跳转表中每个表项给出相应系统调用对应的系统调用服务例程的首地址。

表 7.3 给出了部分 Linux 系统调用的调用号、名称及其含义。

表 7.3 部分 Linux 系统调用示例

调用号	名称	类别	含义	调用号	名称	类别	含义
1	exit	进程控制	终止进程	12	chdir	文件系统	改变当前工作目录
2	fork	进程控制	创建一个新进程	13	time	系统控制	取得系统时间
3	read	文件操作	读文件	19	lseek	文件系统	移动文件指针
4	write	文件操作	写文件	20	getpid	进程控制	获取进程号
5	open	文件操作	打开文件	37	kill	进程通信	向进程或进程组发信号
6	close	文件操作	关闭文件	45	brk	内存管理	修改虚拟空间中的堆指针 brk
7	waitpid	进程控制	等待子进程终止	90	mmap	内存管理	建立虚拟页面到文件片段的映射
8	create	文件操作	创建新文件	106	stat	文件系统	获取文件状态信息
11	execve	进程控制	运行可执行文件	116	sysinfo	系统控制	获取系统信息

因为内核实现的系统调用是以一个软中断的形式（即陷阱指令，如 `int $0x80`）来提供的，如果高级语言编写的用户程序直接用陷阱指令来调用系统调用，则会很麻烦，因此，需要将系统调用封装成用户程序能直接调用的函数，如 `exit()`、`read()` 和 `open()`，这些都是标准 C 库中系统调用对应的封装函数。在用 C 语言编写的用户程序中，只要用 `#include` 命令嵌入相应的头文件，就可以直接使用这些函数来调出操作系统内核中相应的系统调用服务例程，以完成与 I/O、文件操作以及进程管理等相关的操作。在本书中将系统调用及对应的封装函数称为系统级函数。

从 C 语言编程者角度来看，系统级函数在形式上与普通的应用编程接口（API）以及普通的 C 语言函数没有差别。但是，实际上它们在机器级代码的具体实现上是不同的。例如，在 IA-32/Linux 中，普通函数（包括 API）使用 `CALL` 指令来实现过程调用，而系统调用则使用陷阱指令（如 `int $0x80` 或 `sysenter`）来实现。对于过程调用，执行 `CALL` 指令前后，处理器一直在用户态下执行指令，因而，所执行的指令是受限的，所能访问的存储空间也是受限的；而对于系统调用，一旦执行了发出系统调用的陷阱指令，处理器就从用户态转到内核态下运行，此时，CPU 可以执行特权指令并访问内核空间。

普通的 API 以及普通库函数可能会使用一个或多个系统调用服务功能，也可能不需要使用系统调用服务功能，例如，对于数学库函数，就无需使用系统调用服务功能。

在 IA-32/Linux 系统中，系统调用所用的参数通过寄存器传递，而不是像过程调用那样用栈来传递，因此，在封装函数中将使用传送指令把系统调用所使用的参数传送到相应的寄存器。按照惯例，系统调用号存放在 `EAX` 中，传递参数的寄存器顺序依次为：`EAX`（调用号）、`EBX`、`ECX`、`EDX`、`ESI`、`EDI` 和 `EBP`，除调用号以外，最多 6 个参数。若参数个数超出寄存器个数，则将参数块所在内存区首址放在寄存器中传递。

封装函数对应的机器级代码有一个统一的结构：总是若干条传送指令后跟上一条陷阱指令。传送指令用来传递系统调用所用的参数，陷阱指令（如 `int $0x80`）用来陷入内核进行处理。

例如，若用户程序希望将字符串 `"hello, world!\n"` 中的 14 个字符显示在标准输出设备文件 `stdout` 上，则可以调用系统调用 `write(1, "hello, world!\n", 14)`，它的封装函数用以下机器级代码（用汇编指令表示）实现。

```

movl $4, %eax      // 调用号为 4, 送 EAX
movl $1, %ebx      // 标准输出设备 stdout 的文件描述符为 1, 送 EBX
movl $string, %ecx  // 字符串 "hello, world!\n" 的首地址等于 string 的值, 送 ECX
movl $14, %edx     // 字符串的长度为 14, 送 EDX
int $0x80          // 系统调用

```

在 Linux 中, 有一个系统调用的统一入口, 即是系统调用处理程序 `system_call` 的首地址, 所以, CPU 执行指令 `int $0x80` 后, 便转到 `system_call` 的第一条指令开始执行。在 `system_call` 中, 将根据调用号跳转到当前系统调用对应的系统调用服务例程去执行。`system_call` 执行完后返回到 `int $0x80` 指令后面一条指令继续执行。返回参数在 EAX 中, 为整数值, 若是正数或 0 表示成功, 负数表示出错码。

Intel 从 Pentium II 处理器开始, 引入了指令 `sysenter` 和 `sysexit`, 分别用于进入系统调用和退出系统调用。在 Intel 文档中, `sysenter` 被称为快速系统调用指令, 它提供了从用户态到内核态的快速切换方式。对于 Pentium II 以后的 IA-32 体系结构, 在 Linux 和 Windows 等系统中, 都可以通过两种方式发出或退出系统调用。

在 Linux 系统中, 可以通过执行指令 `int $0x80` 或 `sysenter` 来发出系统调用; 相应地, 可以通过执行指令 `iret` 或 `sysexit` 退出系统调用。在 Windows 系统中, 可以通过执行指令 `int $0x2e` 或 `sysenter` 来发出系统调用; 相应地, 可以通过执行指令 `iret` 或 `sysexit` 退出系统调用。

以下给出在 Linux 系统中进入和退出系统调用的大致过程。

1. 通过软中断指令进入和退出系统调用

CPU 在用户空间执行软中断指令 `int $0x80` 的过程与 7.2.7 节描述的异常和中断响应过程一样, CPU 的运行状态从用户态切换为内核态; 并从任务状态段 TSS 中将内核态对应的栈段寄存器内容和栈指针装入 SS 和 ESP; 再依次将原先执行完软中断指令 `int $0x80` 时的栈段寄存器 SS、栈指针 ESP、标志寄存器 EFLAGS、代码段寄存器 CS、指令计数器 EIP 的内容 (即返回地址或断点) 保存到内核栈中, 即当前 SS:ESP 所指之处; 然后从中断描述符表 IDT 中的第 128 (80H) 个表项中取出相应的门描述符 `IDTi (i = 128)`, 将其中的段选择符装入 CS, 偏移地址装入 EIP, 这里, CS:EIP 即是系统调用处理程序 `system_call` 的第一条指令的逻辑地址。需要从系统调用返回时, 则通过执行 `iret` 指令实现上述逆过程。

2. 通过快速系统调用指令进入和退出系统调用

因为系统调用属于陷阱类异常, 所以通过软中断指令 `INT n` 进入和退出系统调用的处理过程, 就是 7.2.7 节中描述的异常和中断的响应过程, 需要进行一连串的一致性和安全性检查, 因而速度较慢。

快速系统调用指令 `sysenter` 主要用于从用户态到内核态的快速切换。为了实现快速系统调用, Intel 在 Pentium II 以后的处理器中增加了以下三个特殊的MSR 寄存器。

SYSTEM_CS_MSR: 存放内核代码段的段选择符。

SYSTEM_EIP_MSR: 存放内核中系统调用处理程序的起始地址。

SYSTEM_ESP_MSR: 存放内核栈的栈指针。

执行 `sysenter` 指令时, CPU 将 SYSTEM_CS_MSR、SYSTEM_EIP_MSR 和 SYSTEM_ESP_MSR 的内容分别复制到 CS、EIP 和 ESP, 同时将 SYSTEM_CS_MSR 的内容加 8 的值设定到 SS。因

此，CPU 执行完 `sysenter` 指令，即可切换到内核态，并开始执行系统调用处理程序的第一条指令。

MSR 寄存器的内容只能通过特权指令 `rdmsr` 和 `wrmsr` 进行读写。

7.3 小结

进程是一个具有一定独立功能的程序关于某个数据集的一次运行活动，每个进程都有其独立的逻辑控制流和私有的虚拟地址空间。每个进程的逻辑控制流是确定的，不管这个逻辑控制流在哪个指令地址处被打断。每个被打断的逻辑控制流处都发生了一个异常控制流。造成这种异常控制流的原因有多种，可能是操作系统进行进程的处理器调度时引起的，可能是硬件在执行指令时检测到有异常或中断事件时引起的，还可能是一个进程利用信号机制向另一个进程发送信号而引起的。异常控制流是并发执行的基本机制。

操作系统通过进程的处理器调度，将当前正在处理器上执行的一个进程换下，把另一个进程换上处理器执行，使得系统在当前进程的执行过程中发生了一个异常控制流，这种异常控制流通过进程的上下文切换来实现。

硬件在执行指令过程中，会随时监测有无异常发生。当发现当前执行的是陷阱指令，或有异常发生、有外部中断请求时，则当前进程发生异常控制流，转入一个特定的内核程序执行，以针对特定的异常或中断进行处理，这个内核程序执行的代码不是一个进程，而是一个“内核控制路径”，它代表当前进程在内核态执行单独的一个指令序列。

对于不同类型的异常或中断，其处理方式可能不同。对于陷阱指令，则相当于提供了一个过程调用，在陷阱指令执行结束后转入到一个特定的内核程序执行，执行结束后返回到陷阱指令的后面一条指令继续执行；对于无法恢复的故障类异常，如非法操作码、除法错、越界（或越权、越级）类访存错等，则相应的异常处理程序会向当前进程发送一个特定的信号，当前进程接收到信号后，就调用相应的信号处理程序执行（如果有对应的信号处理程序的话），或调用内核的 `abort` 例程终止当前进程（如果没有对应的信号处理程序的话）；对于可以恢复的故障类异常，则相应的异常处理程序处理完故障后，会回到当前进程的故障指令继续执行；对于外部中断，则在相应的中断服务程序执行后，回到当前进程的下一条指令继续执行。

习题

1. 给出以下概念的解释说明。

CPU 的控制流	正常控制流	异常控制流	进程	逻辑控制流
物理控制流	并发 (concurrency)	并行 (parallelism)	多任务	时间片
进程的上下文	系统级上下文	用户级上下文	寄存器上下文	进程控制信息
上下文切换	内核空间	用户空间	内核控制路径	异常处理程序
中断服务程序	故障 (fault)	陷阱 (trap)	陷阱指令	终止 (abort)
中断请求信号	中断响应周期	中断类型号	开中断/关中断	可屏蔽中断

不可屏蔽中断	断点	程序状态字寄存器	程序状态字	向量中断方式
中断向量	中断向量表	异常/中断查询程序	中断描述符表	I/O 中断
时钟中断	处理器中断	现场信息	中断服务程序	中断服务例程
系统调用	系统调用号	系统调用处理程序	系统调用服务例程	

2. 简单回答下列问题。

- (1) 引起异常控制流的事件主要有哪几类?
- (2) 进程和程序之间最大的区别在哪里?
- (3) 进程的引入为应用程序提供了哪两个方面的假象? 这种假象带来了哪些好处?
- (4) “一个进程的逻辑控制流总是确定的, 不管中间是否被其他进程打断, 也不管被打断几次或在哪里被打断, 这样, 就可以保证一个进程的执行不管怎么被打断其行为总是一致的。” 计算机系统主要靠什么机制实现这个能力?
- (5) 在进行进程上下文切换时, 操作系统主要完成哪几项工作?
- (6) 在 IA-32/Linux 系统平台中, 一个进程的虚拟地址空间布局是怎样的?
- (7) 简述异常和中断事件形成异常控制流的过程。
- (8) 调试程序时的单步跟踪是通过什么机制实现的?
- (9) 在异常和中断的响应过程中, CPU(硬件) 要保存一些信息, 这些信息包含哪些内容?
- (10) 在执行异常处理程序和中断服务程序过程中, (软件) 要保存一些信息, 这些信息包含哪些内容?
- (11) 普通的过程(函数)调用和操作系统提供的系统调用之间有哪些相同之处? 有哪些不同之处?
- (12) 在 IA-32 中, 中断向量表和中断描述符表各自记录了什么样的信息?

3. 根据下表给出的 4 个进程运行的起、止时刻, 指出每个进程对 P1 - P2、P1 - P3、P1 - P4、P2 - P3、P3 - P4 中的两个进程是否并发运行?

进程	开始时刻	结束时刻
P1	1	7
P2	4	6
P3	3	8
P4	2	5

4. 假设在 IA-32/Linux 系统中一个 main 函数的 C 语言源程序 P 如下:

```

1 unsigned short b[2500];
2 unsigned short k;
3 main()
4 {
5     b[1000] = 1023;
6     b[2500] = 2049*k;
7     b[10000] = 20000;
8 }
```

经编译、链接后, 第 5、6 和 7 行源代码对应的指令序列如下:

```

1 movw    $0x3ff, 0x80497d0    // b[1000] = 1023
2 movw    0x804a324, %cx       // R[cx] = k
```

```

3 movw    $0x801, %ax           // R[ax] = 2049
4 xorw    %dx, %dx             // R[dx] = 0
5 div     %cx                  // R[dx] = 2049%k
6 movw    %dx, 0x804a324        // b[2500] = 2049%k
7 movw    $0x4e20, 0x804de20    // b[10000] = 20000

```

假设系统采用分页虚拟存储管理方式，页大小为 4KB，第 1 行指令对应的虚拟地址为 0x80482c0，在运行 P 对应的进程时，系统中没有其他进程在运行，回答下列问题。

- (1) 对于上述 7 条指令的执行，是否可能在取指令时发生缺页故障？
 - (2) 执行第 1、2、6 和 7 行指令时，在访问存储器操作数的过程中是否会发生页故障或其他什么问题？哪些指令中的问题是可恢复的？哪些指令中的问题是不可恢复的？分别画出第 1 行和第 7 行指令所发生故障的处理过程示意图。
 - (3) 执行第 5 条指令时会发生什么故障？该故障能否恢复？
5. 若用户程序希望将字符串“hello, world!\n”中的 14 个字符显示在标准输出设备文件 stdout 上，则可以使用系统调用 write 对应的封装函数 write(1, “hello, world!\n”, 14)，在 IA-32/Linux 系统中，可以用以下机器级代码（用汇编指令表示）实现。

```

1 movl    $4, %eax             // 调用号为 4, 送 EAX
2 movl    $1, %ebx             // 标准输出设备 stdout 的文件描述符为 1, 送 EBX
3 movl    $string, %ecx        // 字符串“hello, world!\n”的首地址等于 string 的值, 送 ECX
4 movl    $14, %edx            // 字符串的长度为 14, 送 EDX
5 int     $0x80                // 系统调用

```

针对上述机器级代码，回答下列问题或完成下列任务。

- (1) 执行该段代码时，系统处于用户态还是内核态？为什么？执行完第 5 行指令后的下一个时钟周期，系统处于用户态还是内核态？
 - (2) 第 5 行指令是否属于陷阱指令？执行该指令时，通过 5 种类型（中断门、系统门、系统中断门、陷阱门和任务门）门描述符中的哪种来激活异常处理程序？对应的中断类型号是多少？对应门描述符中的字段 P、DPL、TYPE 的内容分别是什么？根据对应门描述符中的段选择符取出的 GDT 中的段描述符中的基地址、限界、字段 G、S、TYPE（包含 A）、DPL、D 和 P 分别是什么？
 - (3) 参考 7.2.7 节的内容，详细描述第 5 行指令的执行过程。
6. IA-32 和 Linux 分别代表了硬件和软件（操作系统内核），根据它们各自在整个异常和中断处理过程中所做的具体工作，归纳总结出硬件和软件在异常和中断处理过程中分别完成哪些工作。